NLPApp

Offline Text Studio

College Mini Project Report

Python desktop application for local natural language processing

Prepared from the implemented repository on 27 October 2026.

Repository: https://github.com/amaansayed184/Project-2-NLPApp

Abstract

NLPApp brings six text-processing tasks into one desktop application: sentiment analysis, named entity recognition, extractive summarization, spelling and basic grammar correction, language detection, and extractive headline generation. The original application used NLP Cloud with embedded API keys. The revised version uses local libraries and algorithms so analysis works without cloud requests or usage fees after a one-time installation.

A Tkinter interface accepts multiline text and UTF-8 files, runs analysis in a background worker, and provides copy/save controls. Optional local profiles use salted password hashes. The system demonstrates integration of NLP techniques, desktop programming, validation, local storage, testing and GitHub distribution. It integrates existing pretrained resources; it does not train a new model.

Report map

- Page 2: Problem, objectives and scope
- Page 3: Architecture and module design
- Page 4: Sentiment, entities and language detection
- Page 5: Summaries, spelling and headlines
- Page 6: Data handling, security and usability
- Page 7: Installation on another computer
- Page 8: Demonstration and test evidence
- Page 9: Evaluation, viva, future work and references

Add your institution's cover sheet, student names, roll numbers, department, guide approval and signatures according to its submission format. These details have not been fabricated.

1. Problem, objectives and scope

Problem statement

Cloud-based text applications require connectivity, valid credentials and service availability. Those dependencies can interrupt a classroom demonstration and complicate setup on another computer. The original implementation also repeated API-client setup and stored account passwords as plain text.

The mini project addresses these issues by integrating local NLP components into a desktop workspace. Installation still requires internet to acquire dependencies and the English model; the analysis stage does not.

Objectives

- Provide six common text-processing tools in one interface.
- Remove API keys, paid inference and runtime cloud requests.
- Process text locally on a CPU without a GPU requirement.
- Improve multiline input, file loading, output readability and export.
- Preserve optional registration while hashing stored passwords.
- Distribute source, dependencies, setup helpers, demo data and tests through GitHub.

Proposed system

Users launch the application, sign in or continue as guests, select a task, and enter text or load a sample file. Input is validated before one background job executes. Results appear in a readable output area and can be copied or saved. Resources are loaded when first needed and reused within the running application.

Scope and contribution

The project combines pretrained libraries with project-written sentence ranking, headline extraction, explicit grammar rules, validation, account storage and desktop integration. Most tasks target English; language detection supports the languages recognized by its library. Accuracy on arbitrary text is not guaranteed.

Deliverables

The repository contains runnable source, dependency manifests, setup and verification helpers, a Windows launcher, sample texts, functional tests, a GitHub Actions workflow, a detailed README, a demonstration/viva guide and this report with editable source.

2. Architecture and module design

Processing flow

```
User text / UTF-8 file
         |
         v
Tkinter workspace (app.py)
         |
Input validation and tool options
         |
         v
One background analysis worker
         |
         v
LocalNLP engine (nlp_engine.py)
  VADER | spaCy | langdetect
  SymSpell | sentence ranking
         |
         v
Main-thread result display --> Copy / Save

Optional account flow:
Login / Register --> mydb.py --> local db.json
```

Module responsibilities

- app.py: forms, tool selection, multiline editors, counters, file operations and result formatting.
- nlp_engine.py: independent analysis methods and cached local resources.
- mydb.py: account validation, duplicate detection, hashing, login and legacy migration.
- api_handling.py: compatibility alias from the earlier API class name to LocalNLP; no remote calls.
- scripts/: setup, installation verification and optional report generation.
- tests/: offline functional checks and account-storage tests.

Background execution and storage

ThreadPoolExecutor uses one worker. Tkinter polls completion with root.after on the main thread. Screen identifiers prevent a completed job from writing results into a later screen after navigation. A running job is allowed to finish; there is no interactive cancellation control.

The sentiment analyzer, entity model and spelling dictionary load lazily and are cached. Text stays in memory unless exported. Only account records are automatically persisted. There is no remote database or web server.

3. NLP methodology: classification and entities

Sentiment analysis

VADER applies a supplied sentiment lexicon and rule-based analyzer. It returns a compound score and positive, neutral and negative proportions. The interface maps compound >= 0.05 to positive, <= -0.05 to negative, and intermediate scores to neutral. The compound score ranges from -1 to +1; category proportions are not confidence estimates. These conventions follow VADER's documented scoring [1].

Sentence-level evidence now supplements the whole-text score. A Mixed label is used when both positive (>= 0.3) and negative (<= -0.3) sentences occur, each contributes at least 25% of polarized sentences, and the overall compound magnitude is below 0.5. This is an explicit heuristic, not a trained classifier. Sarcasm and non-English text can still be misread.

Named entity recognition

The pretrained spaCy en_core_web_sm English pipeline predicts spans such as people, organizations, places and dates. The engine returns entity text, label and character offsets. The interface displays readable descriptions. An optional filter matches entity text by case-insensitive substring or matches an exact label such as PERSON.

Example: Barack Obama visited London on Monday. This provides a person, a location and a date for demonstration. Predictions are not factual verification. The model is downloaded during installation and runs locally afterward [2].

Language detection

langdetect uses its supplied language profiles to estimate language codes and probabilities. DetectorFactory.seed is fixed to zero for repeatable results on the same installation and input [3]. The application rejects passages containing fewer than ten alphabetic characters.

A longer English paragraph should typically return en. Short or ambiguous passages now receive a warning. Short, transliterated or mixed-language inputs can be unreliable; high estimated probability does not guarantee correctness. Entity filters also accept readable categories such as people, location and company.

Training and datasets

The project uses existing lexicons, language profiles and a pretrained entity model. Demo files are examples, not a training dataset or benchmark. No custom training, fine-tuning, accuracy percentage or corpus evaluation result is claimed.

4. NLP methodology: transformation and extraction

Extractive summarization

The summarizer uses spaCy's trained parser for sentence boundaries and its lemmas for English words. It removes stopwords and duplicate sentence signatures, then builds normalized sparse TF-IDF vectors. This replaces the earlier frequency-only baseline.

```
TF = 1 + log(term count)
IDF = 1 + log((1 + sentence count) / (1 + document frequency))
relevance = cosine(sentence vector, document centroid)
selection = relevance - 0.45 * maximum selected-sentence overlap
```

Relevance also includes a small lead-sentence bonus and a penalty for very short content. Greedy selection penalizes cosine overlap with already-selected sentences. It selects up to the requested one to ten unique sentences and restores source order. The sparse method avoids a quadratic all-pairs matrix. It can still lose context; it is not TextRank or a generative model.

Spelling and basic grammar

SymSpellPy loads its bundled English dictionary. Unknown longer words allow edit distance two; shorter words allow one. Suggestions need adequate dictionary frequency and a clear frequency advantage over alternatives. Common typos receive explicit fixes, including capitalized Teh. Names and acronyms are otherwise preserved [4].

Protected spans preserve URLs, emails, code, Windows paths and numbered tokens. Prose-only rules repair selected missing apostrophes, pronoun agreement, repeated function words and spaces before punctuation. Corrections remain suggestions, not comprehensive grammar analysis.

Verified example: i is happy with teh speling. becomes I am happy with the spelling. This is basic correction, not a comprehensive grammar checker. Uncommon valid words may be changed, so users must review suggestions.

Extractive headline generation

The tool uses the same sentence ranking, prefers a complete short clause when possible, and otherwise retains at most twelve words while avoiding trailing function words such as the or of. It strips punctuation and marks truncation with an ellipsis. It does not paraphrase; manual editing may still be needed.

Algorithmic contribution

The project-written extraction methods make the workflow explainable without training infrastructure. They trade linguistic sophistication for simplicity. Improvements should be assessed with a defined evaluation dataset rather than unmeasured quality claims.

5. Data handling, security and usability

Optional local accounts

Guest mode exposes all analysis tools without registration. Accounts validate a name, email format and a password of at least eight characters. Email matching is case-insensitive and duplicate registration is rejected.

Passwords are stored using PBKDF2-HMAC-SHA256 with a random sixteen-byte salt and 600,000 iterations. Records contain a hash, salt, iteration count and name. Login derives a hash and compares it with hmac.compare_digest. Legacy plaintext records migrate at startup while preserving the password.

Writes use a temporary file followed by replacement. Damaged JSON raises an error instead of silently overwriting accounts. This local profile store is not designed as a simultaneous multi-process database, encryption at rest, or protection against an administrator who controls the computer.

Privacy and repository hygiene

Names and emails remain readable in local db.json. Inputs and outputs are not automatically logged or saved. Exports and clipboard contents are managed by the user and operating system.

Git ignores db.json, .venv, IDE settings, compiled caches and environment files. New clones start without existing local accounts. Earlier history contained credentials and account records; latest-file removal does not erase old commits. Exposed keys must be revoked and reused passwords changed. History rewriting has not been performed.

Usability and limits

The interface has multiline editing, counters, file loading, copy/save controls, descriptions and background execution. Input is capped at 50,000 characters; imported files must not exceed 200,000 bytes. The first resource load can take longer than subsequent analysis.

Environment requirements

Use a supported 64-bit Python installation with pip and Tkinter, a graphical desktop, writable folders and an ordinary CPU. Python 3.12 is recommended for a consistent new setup; no GPU is required. Initial installation needs internet. Minimum RAM and throughput have not been benchmarked.

6. Installation on another computer

Prepare before college

On the laptop that will actually be used in college, install Git and 64-bit Python 3.12, including pip, Tcl/Tk and the Python launcher. With internet available, open PowerShell in a writable folder:

```
git clone https://github.com/amaansayed184/Project-2-NLPApp.git
cd Project-2-NLPApp
.\setup.bat
.\run.bat
```

Setup creates .venv, installs the libraries and English model, and checks all six tools with networking blocked. Wait for Setup complete, then launch run.bat and choose Continue as guest. If Git is unavailable, download the GitHub ZIP, extract it completely and run setup.bat inside the extracted folder.

Manual alternative

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-model.txt
.\.venv\Scripts\python.exe scripts\verify_setup.py
.\.venv\Scripts\python.exe app.py
```

Using the environment's Python directly avoids activation and execution-policy changes.

Confirm offline readiness

Cloning supplies source code, not Python, installed libraries or the entity model. Do not copy .venv between computers; recreate it. A source ZIP alone is not an offline installer. If the laptop will lack internet at college, finish installation on that same laptop beforehand.

```
.\.venv\Scripts\python.exe scripts\verify_setup.py
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Then disconnect Wi-Fi and manually try every feature. The helper imports Tkinter but does not open a window, so the manual GUI check matters. Accounts do not transfer through Git; use guest access or a new local account. The README provides updates, macOS/Linux commands and troubleshooting.

7. Demonstration and test evidence

Suggested demonstration sequence

- Launch run.bat and continue as guest. Introduce local processing.
- Sentiment: load demo/sentiment.txt and show its positive result.
- Entities: load demo/entities.txt; show predicted spans and try PERSON as a filter.
- Summary: load demo/article.txt, request two sentences and compare with the source.
- Spelling: load demo/spelling.txt and review its corrections.
- Language: load demo/language.txt and explain the en code.
- Headline: load demo/article.txt, then show copy/save controls.
- Demonstrate continued analysis with Wi-Fi disconnected after setup.

Automated test evidence

All 17 unittest methods passed locally after the quality improvements. The original five cover:

- All six tools with socket connections patched to fail, including known positive/negative examples, entity recognition/filtering, English detection, spelling correction, extractive summary membership and headline length.
- Blank and oversized input, and unsuitable language-detection input.
- Registration, duplicate emails, correct/incorrect login, case-insensitive email matching and different salted hashes for identical passwords.
- Legacy account migration with preserved login, plus damaged-database errors without overwriting the damaged file.

A separate Windows desktop smoke check passed for the guest workspace, background analysis, result display and navigation during a running task. The setup verifier additionally imports Tkinter and runs six sample jobs with socket connections blocked.

Twelve new regressions cover duplicate-free summaries, abbreviations/decimals, extractive source order, input counts, complete-clause headlines, common typos, protected technical text/code, agreement/contractions, mixed sentiment, language warnings and readable entity filters. They are targeted examples, not independent accuracy measurements.

The setup helper was also tested in a separate clean project copy on Windows with Python 3.12.0. It created a new environment, installed the libraries/model and passed all six offline verification jobs and all five tests.

Interpretation and reproducibility

These are functional checks, not NLP accuracy benchmarks, performance measurements or proof of zero network activity under every possible execution path. GitHub Actions installs the libraries/model and executes the suite for pushes and pull requests. Inspect the Actions tab for a particular run's status.

The destination laptop needs its own setup, verification and manual offline GUI check. The demo/viva guide provides a presentation script and sample explanations.

8. Evaluation, viva and future work

Outcome and limitations

The application replaces runtime API dependence with six local processing paths, improves desktop interaction and hashes local passwords. Its trade-off is quality: lightweight extraction and rules do not match every capability of a large generative model. Most tasks assume English. Entity/language predictions can be wrong; grammar rules and sentence splitting are limited. No dataset accuracy or minimum hardware specification is asserted.

Future work

Potential extensions include labeled-corpus evaluation, better sentence segmentation, multilingual pipelines, user-reviewed corrections, accessibility, job cancellation and a tested packaged installer. Larger local models would require storage and latency evaluation. These are proposed improvements, not current features.

Viva essentials

- Why local NLP? It removes runtime credentials, quotas and connectivity dependencies while processing input on the computer.
- Did you train a model? No. The project integrates existing resources with its own UI, storage, ranking and rule logic.
- Extractive versus abstractive? Extraction selects source text; abstraction generates rewritten text. This project uses extraction.
- Why a background worker? It keeps analysis off Tkinter's event loop; widgets update on the main thread.
- Is internet never needed? First installation and future dependency downloads need internet; normal analysis does not.
- How are passwords protected? Salted PBKDF2 hashes replace plaintext; this is still a local profile system, not a hardened multi-user service.

Primary references

[1] VADER scoring and project: https://github.com/cjhutto/vaderSentiment

[2] spaCy models and pipelines: https://spacy.io/usage/models

[3] langdetect usage and seed: https://pypi.org/project/langdetect/

[4] SymSpellPy lookup: https://symspellpy.readthedocs.io/en/latest/examples/lookup.html

Source, installation instructions and tests: https://github.com/amaansayed184/Project-2-NLPApp